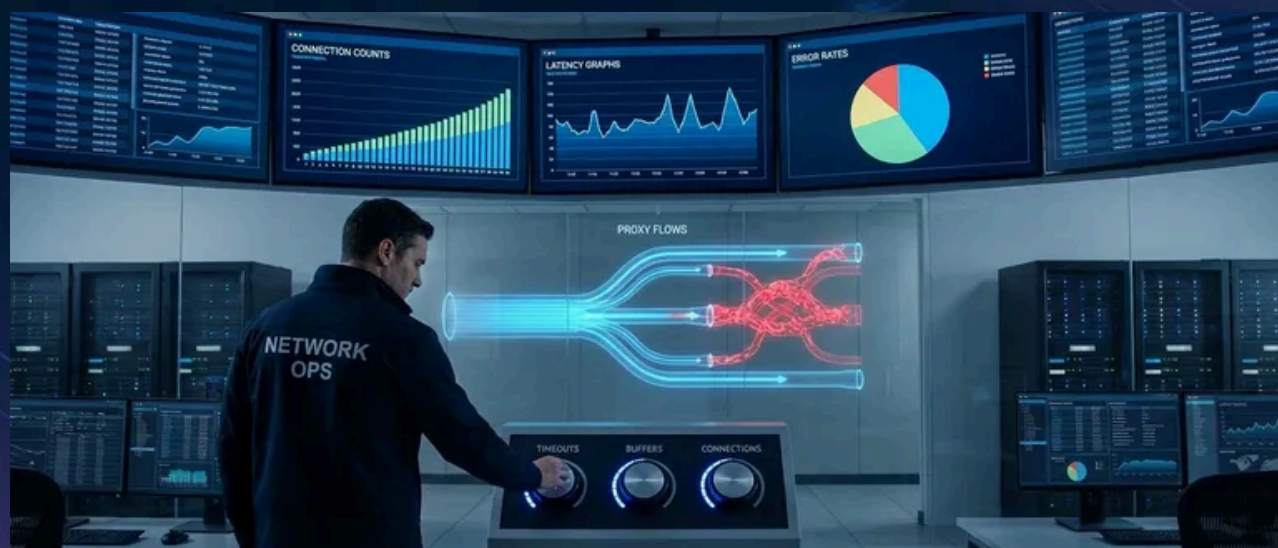


Reverse Proxy Hardening: Timeouts, Buffers, Defaults



Published on August 17, 2024



Webstack
Builders

Table of Contents

Understanding Proxy Architecture	3
Request Flow Through Proxies	3
Connection Multiplexing	5
Timeout Configuration	7
Nginx Timeout Hierarchy	7
HAProxy Timeout Hierarchy	9
Buffer Tuning	12
Nginx Buffer Configuration	12
HAProxy Buffer Configuration	14
Connection Management	16
Keep-Alive Tuning	16
HAProxy Connection Pooling	18
Load Balancing Configuration	20
Algorithm Selection	20
HAProxy Advanced Load Balancing	22
SSL/TLS Optimization	24
Certificate and Protocol Configuration	24
HAProxy SSL Configuration	26
Monitoring and Observability	29
Nginx Metrics and Logging	29
HAProxy Statistics	31
Alerting on Proxy Metrics	32
Production Hardening Checklist	34
Security Configuration	34
Complete Production Config Example	36
Applying Configuration Changes	40
Conclusion	40



Nginx and HAProxy ship with defaults optimized for getting started quickly, not for handling production traffic. Default buffer sizes assume small requests. Default timeouts assume fast backends. Default connection limits assume modest traffic. When real load arrives—10,000 concurrent connections, slow clients on mobile networks, backends that occasionally take 30 seconds to respond — these defaults fail in ways that are hard to diagnose.

A 502 error from Nginx could mean a dozen different things. The backend refused the connection. The backend accepted the connection but didn't respond in time. The backend started responding but the response was too large to buffer. The proxy ran out of file descriptors. Without understanding the proxy's internals, you're guessing.

Here's a scenario I've seen play out. A team deploys their API behind Nginx with the default configuration. Traffic grows to 5,000 RPS (Requests Per Second). Intermittent 502 errors start appearing. The backend looks healthy — response times are fine, no errors in application logs. The problem is invisible until someone digs into Nginx's internals. It turns out `proxy_read_timeout` defaults to 60 seconds, which sounds generous until you realize a few slow endpoints sometimes take 65 seconds (database queries, external API calls). Meanwhile, `worker_connections` is 1024, and with keep-alives holding connections open, they're running out of connection slots during traffic spikes.

After tuning timeouts, increasing worker connections, and adjusting buffer sizes, the 502s disappear. The lesson: proxy configuration is where traffic patterns meet system limits. Production tuning isn't optional optimization — it's the difference between a proxy that handles traffic gracefully and one that drops connections under load.

WARNING

The most dangerous proxy configuration is one that works perfectly in development. Production traffic patterns — slow clients, large payloads, connection storms — expose every untuned default.

Understanding Proxy Architecture

Request Flow Through Proxies

Every HTTP request passing through a reverse proxy goes through distinct phases, each with its own timeout and buffer settings. Understanding this flow is essential for diagnosing where failures occur.



The journey starts with the **client connection phase**: TCP (Transmission Control Protocol) handshake, then TLS (Transport Layer Security) handshake if HTTPS. This is where `listen` and `ssl_protocols` in Nginx (or `bind` and `ssl-min-ver` in HAProxy) come into play. A slow TLS (Transport Layer Security) handshake here – perhaps due to missing OCSP (Online Certificate Status Protocol) stapling or expensive cipher negotiation – adds latency before the proxy even sees the HTTP request.

Next comes **request receipt**: the proxy reads the request line, headers, and body. This is where `client_header_timeout` and `client_body_timeout` apply. A mobile client on a flaky connection might take 10 seconds to send a 5MB upload. If your timeout is 5 seconds, the connection drops.

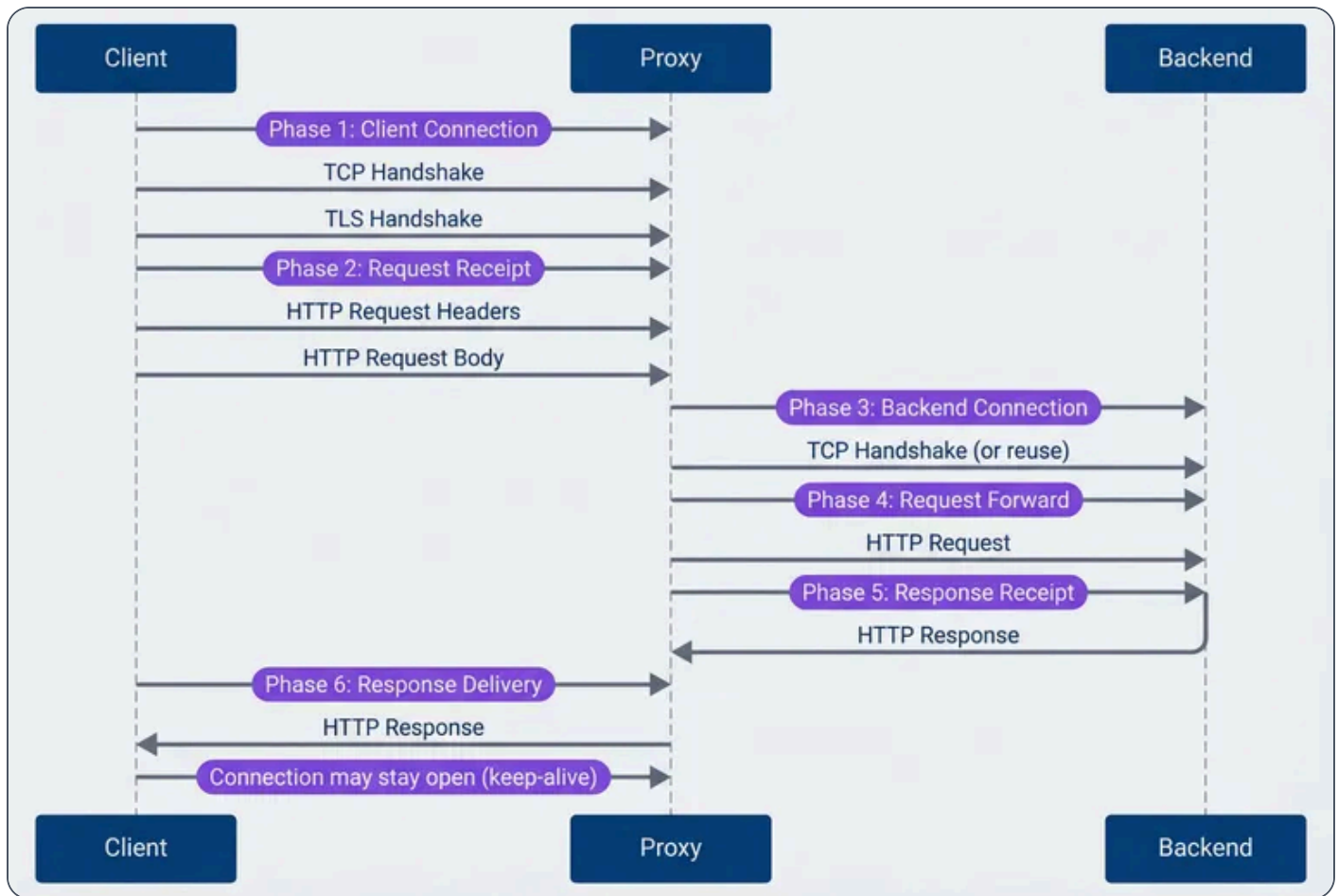
The **backend connection phase** is often the source of 502 errors. The proxy opens a connection to the upstream server (or reuses one from a keep-alive pool). These backend connections can be TCP (Transmission Control Protocol) sockets, Unix domain sockets (for backends on the same host), or TLS (Transport Layer Security)-encrypted connections for backends that require encryption in transit.

In cloud deployments, the pattern varies: within a Kubernetes cluster, backends typically run on the same node or within a private network, so plain HTTP over TCP (Transmission Control Protocol) is common – TLS (Transport Layer Security) termination happens at the ingress, not between services. For backends in different VPCs or external services, you'll often see TLS (Transport Layer Security) to the backend, which adds a TLS (Transport Layer Security) handshake to each new connection (another reason keep-alive pools matter). Unix sockets eliminate network overhead entirely but only work when proxy and backend share a filesystem.

The configuration parameters `proxy_connect_timeout` in Nginx or `timeout connect` in HAProxy controls how long to wait for the backend connection. If the backend is overloaded and not accepting connections, this timeout fires.

Finally, the proxy **forwards the request, receives the response, and delivers it to the client**. Each phase has its own timeout. The response delivery phase is particularly important for slow clients – a backend might respond in 50ms, but if the client is on a 2G connection, sending that response takes seconds.





Request flow through reverse proxy.

Connection Multiplexing

Nginx and HAProxy handle connections differently, and understanding these models helps you size connection limits appropriately.

Nginx uses an **event-driven, single-threaded worker** model. Each worker process handles thousands of connections using non-blocking I/O, but each connection – whether from a client or to a backend – uses one slot from the `worker_connections` pool. Since a proxied request needs both a client connection and a backend connection, each request consumes at least two slots. The formula for maximum concurrent requests is roughly:

$$\text{max concurrent requests} = (\text{worker_processes} \times \text{worker_connections}) / 2$$



With 4 workers and 4096 connections each, you get approximately 8,192 concurrent requests. Keep-alives complicate this – idle connections still consume slots, so you may hit connection limits before CPU or memory becomes a bottleneck.

The `worker_processes auto` setting tells Nginx to spawn one worker per CPU core, which is usually correct. You might deviate for CPU-bound workloads (fewer workers to reduce context switching) or in containers with CPU limits (set workers explicitly since `auto` reads the host's CPU count, not the container's limit).

HAProxy uses an **event-driven, multi-threaded** model with more granular controls. You set a global `maxconn` limit, but you can also set per-frontend and per-backend limits. When a backend reaches its `maxconn`, HAProxy queues requests rather than rejecting them immediately – the `timeout queue` setting controls how long requests wait for an available slot.

HTTP/2 (Hypertext Transfer Protocol Version 2) changes the equation significantly. A single HTTP/2 (Hypertext Transfer Protocol Version 2) connection can multiplex hundreds of concurrent streams (requests). In Nginx, `http2_max_concurrent_streams` defaults to 128; in HAProxy, `tune.h2.max_concurrent_streams` defaults to 100. This means one connection slot serves many requests, dramatically improving efficiency for clients that support HTTP/2 (Hypertext Transfer Protocol Version 2).

HTTP/3 (QUIC) takes this further by eliminating head-of-line blocking entirely – since it runs over UDP, a lost packet doesn't stall unrelated streams. Both Nginx and HAProxy have experimental HTTP/3 support, but production adoption is still early. If you're starting fresh, it's worth evaluating; for existing deployments, HTTP/2 (Hypertext Transfer Protocol Version 2) remains the practical choice.

INFO

If you're proxying gRPC traffic, HTTP/2 (Hypertext Transfer Protocol Version 2) is mandatory – gRPC requires it. Make sure both your frontend (client-facing) and backend connections are configured for HTTP/2 (Hypertext Transfer Protocol Version 2), or gRPC calls will fail with cryptic errors.



Timeout Configuration

Timeouts are the most common source of proxy-related outages, and the defaults are almost never right for production. Nginx defaults most timeouts to 60 seconds – generous enough to hide problems during development, short enough to cause 502s when a backend occasionally takes 65 seconds. HAProxy is worse: many timeouts have **no default**, meaning connections can hang indefinitely if you don't configure them.

The key insight is that timeouts should match your traffic patterns, not arbitrary round numbers. A health check endpoint should respond in milliseconds; a report generation endpoint might legitimately take 5 minutes. Using the same timeout for both means either your health checks are too slow to detect failures, or your reports timeout prematurely.

Nginx Timeout Hierarchy

Nginx organizes timeouts into client-side (receiving requests) and proxy-side (communicating with backends). Most timeouts are **per-operation**, meaning they reset when data flows – a client uploading a large file won't timeout as long as chunks keep arriving.

nginx-timeouts.conf

```
1  # Nginx timeout configuration - production values
2
3  # === Client-side timeouts ===
4
5  # Time to wait for client to send request headers
6  # Default: 60s - Too long for production
7  client_header_timeout 10s;
8
9  # Time to wait for client to send request body
10 # Per-read, not total. Resets on each chunk received.
11 # Default: 60s
12 client_body_timeout 30s;
13
14 # Time to wait between writes to client
15 # If client stops reading, connection closes after this
16 # Default: 60s
17 send_timeout 30s;
18
19 # Keep-alive timeout (client connection reuse)
20 # How long to keep idle connections open
```




```
21  # Default: 75s
22  keepalive_timeout 65s;
23
24  # === Proxy/Backend timeouts ===
25
26  # Time to establish connection to backend
27  # Should be short - backends should be reachable quickly
28  # Default: 60s - Way too long
29  proxy_connect_timeout 5s;
30
31  # Time to wait between writes to backend
32  # If backend stops accepting data, timeout fires
33  # Default: 60s
34  proxy_send_timeout 30s;
35
36  # Time to wait for backend to send response body
37  # Per-read operation, resets on each chunk
38  # Default: 60s - May need increase for slow endpoints
39  proxy_read_timeout 60s;
40
41  # === Per-location overrides ===
42  location /api/slow-endpoint {
43      # Long-running operations need longer timeouts
44      proxy_read_timeout 300s;
45  }
46
47  location /api/uploads {
48      # Large uploads need time
49      client_body_timeout 120s;
50      proxy_send_timeout 120s;
51  }
52
53  location /health {
54      # Health checks should be fast
55      proxy_connect_timeout 2s;
56      proxy_read_timeout 5s;
57  }
```

The per-location overrides are essential. A monolithic timeout configuration forces you to set everything to accommodate the slowest endpoint, which masks problems elsewhere. By setting aggressive defaults (5-second connect, 60-second read) and overriding for specific slow paths, you get fast failure detection for most endpoints while still supporting legitimate long-running operations.



HAProxy Timeout Hierarchy

HAProxy's timeout model differs from Nginx in one critical way: `timeout client` and `timeout server` cover the **entire** request or response, not per-read operations. If you set `timeout server 60s` and the backend takes 30 seconds to send the first byte, then another 35 seconds to send the body, the connection times out – even though data was flowing the whole time.

haproxy-timeouts.cfg

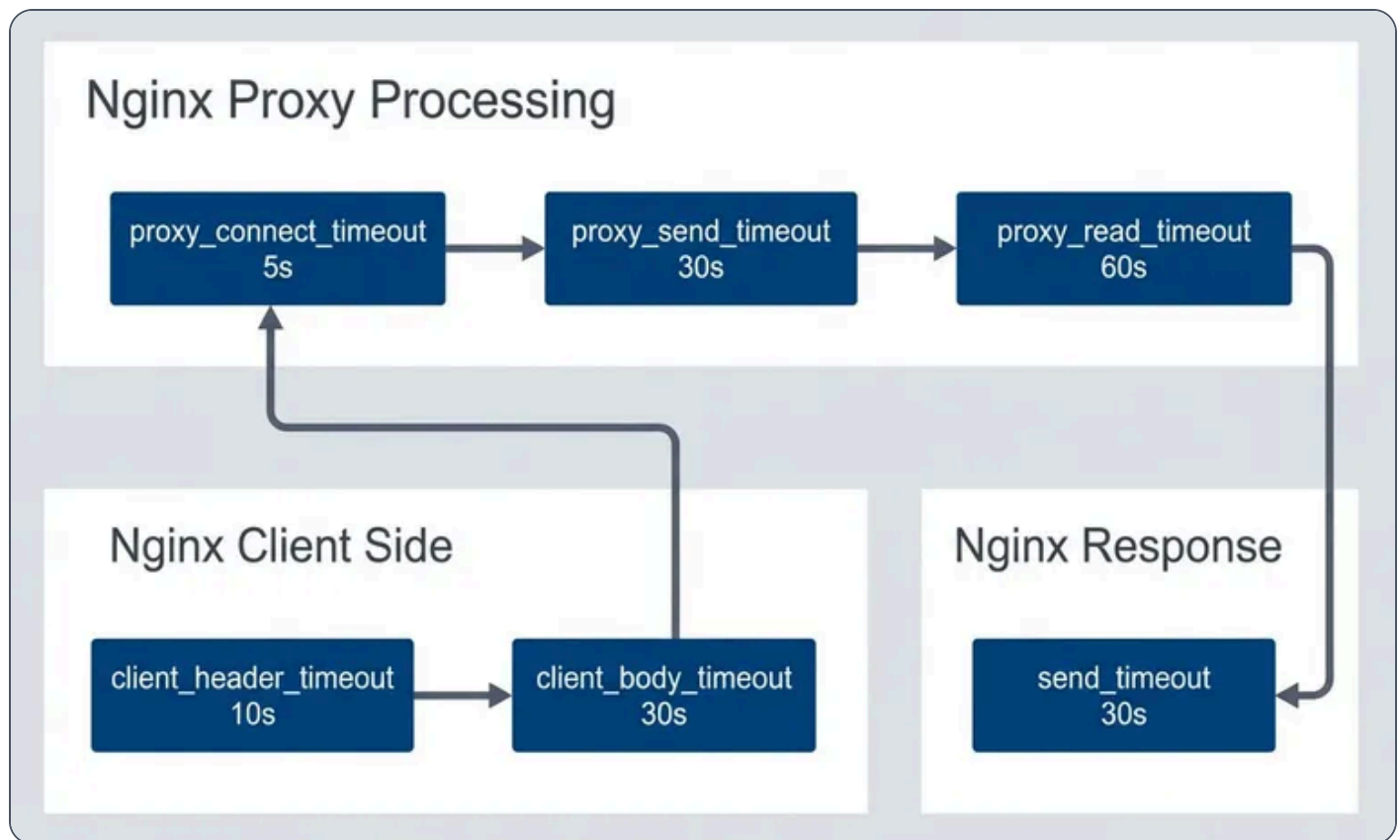
```
1  # HAProxy timeout configuration - production values
2
3  defaults
4      mode http
5
6      # === Connection timeouts ===
7
8      # Time to wait for TCP connection to backend
9      # Default: no timeout (dangerous!)
10     timeout connect 5s
11
12     # Time to wait for client to send data
13     # Covers entire request, not per-read
14     # Default: no timeout
15     timeout client 30s
16
17     # Time to wait for server to send data
18     # Covers entire response, not per-read
19     # Default: no timeout
20     timeout server 60s
21
22     # === HTTP-specific timeouts ===
23
24     # Time to wait for complete HTTP request
25     # Protects against slowloris attacks
26     # Default: 5s
27     timeout http-request 10s
28
29     # Time to wait between requests on keep-alive connection
30     # How long to keep connection open waiting for next request
31     # Default: no timeout
32     timeout http-keep-alive 10s
33
```



```
34      # Time to wait in queue for available server
35      # When all backend servers are at maxconn
36      # Default: no timeout
37      timeout queue 30s
38
39      # Time to wait for health check response
40      # Default: uses timeout connect + timeout server
41      timeout check 5s
42
43      # === Tunnel timeout (websockets, etc.) ===
44      # Time for idle bidirectional connection
45      timeout tunnel 3600s
46
47      # Per-backend overrides
48      backend slow_api
49          timeout server 300s
50          timeout queue 60s
51
52      backend uploads
53          timeout client 120s
54          timeout server 120s
55
56      backend health_checks
57          timeout connect 2s
58          timeout server 5s
```

The `timeout queue` setting deserves special attention. When all backend servers reach their `maxconn` limit, HAProxy queues incoming requests rather than rejecting them immediately. This is usually what you want – a brief spike shouldn't return errors if backends will be available in a few seconds. But if the queue timeout is too long, users wait forever for requests that will eventually fail anyway. 30 seconds is a reasonable default; adjust based on how long users are willing to wait.





Nginx timeout chain.

The following table maps Nginx and HAProxy timeout settings to each other. They're not exact equivalents – Nginx's per-read semantics differ from HAProxy's total-time semantics – but this helps when translating configurations between the two.

#	Timeout	Nginx	HAProxy	Recommended
1	Client connect	N/A (kernel)	N/A (kernel)	Kernel default
2	Client request	client_header_timeout	timeout http-request	10-30s
3	Client body	client_body_timeout	timeout client	30-120s
4	Backend connect	proxy_connect_timeout	timeout connect	3-10s



#	Timeout	Nginx	HAProxy	Recommended
5	Backend response	proxy_read_timeout	timeout server	30-300s
6	Keep-alive idle	keepalive_timeout	timeout http-keep-alive	30-120s

Timeout comparison.

⚠ WARNING

The most common timeout mistake: setting `proxy_read_timeout` too short. A backend that occasionally takes 45 seconds (database query, external API) will cause 502 errors with a 30-second timeout. Know your P99 (99th Percentile) backend latency and add headroom.

Buffer Tuning

Buffers determine how much data your proxy can hold in memory between clients and backends. Get them wrong and you'll see one of two problems: undersized buffers cause 400 Bad Request errors (headers too large) or force disk spillover that kills performance; oversized buffers waste memory and can make a high-traffic proxy OOM (Out of Memory).

The most common buffer-related issue is authentication headers exceeding the default buffer size. Nginx's `client_header_buffer_size` defaults to 1KB, but a request with OAuth tokens, cookies, and correlation headers can easily hit 4-8KB. When headers exceed the primary buffer, Nginx falls back to `large_client_header_buffers` —if that's also too small, the client gets a 400 Bad Request with no useful message.

Nginx Buffer Configuration

Nginx separates client-side buffers (receiving requests) from proxy-side buffers (receiving responses from backends). The proxy buffers are where most tuning happens because response sizes vary wildly.



nginx-buffers.conf

```
1  # === Client request buffers ===
2
3  # Buffer for reading client request headers
4  # Default: 1k - Often too small for cookies/auth headers
5  client_header_buffer_size 4k;
6
7  # Fallback large buffers for oversized headers
8  # number * size - used when header exceeds client_header_buffer_size
9  # Default: 4 8k
10 large_client_header_buffers 8 16k;
11
12 # Maximum request body size
13 # 0 = unlimited (dangerous!)
14 # Default: 1m
15 client_max_body_size 100m;
16
17 # Buffer client request body in memory or disk
18 # on = use proxy_buffer_size first, then disk
19 # off = stream to backend immediately
20 client_body_buffer_size 128k;
21
22 # === Proxy response buffers ===
23
24 # Buffer for first part of response (headers)
25 # Should fit typical response headers
26 # Default: 4k/8k (platform dependent)
27 proxy_buffer_size 8k;
28
29 # Buffers for response body
30 # number * size total buffering capacity
31 # Default: 8 4k/8k
32 proxy_buffers 16 32k;
33
34 # How much can be busy sending to client while reading more
35 # Default: 8k/16k
36 proxy_busy_buffers_size 64k;
37
38 # Maximum size of buffered response
39 # If response exceeds this, streams directly (slower)
40 # Default: 256m
41 proxy_max_temp_file_size 1024m;
42
```



```
43  # === Buffering behavior ===
44
45  # Buffer entire response before sending to client
46  # on = faster for slow clients, uses memory
47  # off = streaming, lower memory, can be slower
48  proxy_buffering on;
49
50  # Don't wait for full buffer before sending
51  # Useful for SSE, streaming responses
52  # location /events {
53  #     proxy_buffering off;
54  # }
55
56  # === Per-location tuning ===
57  location /api/large-response {
58      proxy_buffers 32 64k;
59      proxy_busy_buffers_size 128k;
60  }
61
62  location /api/streaming {
63      proxy_buffering off;
64      proxy_read_timeout 3600s;
65  }
```

The `proxy_buffering` directive controls whether Nginx buffers the entire backend response before sending to the client. With buffering on, a backend can finish quickly and move on to the next request even if the client is slow – the proxy holds the response. With buffering off, the backend connection stays open until the client receives everything. For typical APIs, buffering on. For Server-Sent Events, WebSockets, or streaming responses, buffering off.

HAProxy Buffer Configuration

HAProxy uses a simpler buffer model: `tune.bufsize` controls the buffer size for both requests and responses. Unlike Nginx's array of buffers, HAProxy uses a single buffer per direction. This makes tuning simpler but less flexible – you can't have small request buffers and large response buffers.

haproxy-buffers.cfg

```
1  global
```



```

2      # Size of buffer for request/response
3      # Applied to both directions
4      # Default: 16384 (16k)
5      tune.bufsize 32768
6
7      # Maximum number of headers
8      # Default: 101
9      tune.http.maxhdr 128
10
11     # Maximum size of compressed response
12     tune.comp.maxlevel 5
13
14     # Maximum memory for SSL session cache
15     tune.ssl.cachesize 100000
16
17     # Maximum number of headers to log
18     tune.http.logurilen 8192
19
20     defaults
21         # Maximum request/response size (header + body)
22         # Requests exceeding this get 400 error
23         # Default: tune.bufsize
24         # For large uploads, must increase tune.bufsize
25
26         # Response buffering mode
27         # full: buffer entire response
28         # auto: decide based on response
29         option http-buffer-request
30
31     frontend main
32         # Rate limiting headers/body
33         http-request set-header X-Request-Size %[req.body_size]
34
35         # Reject oversized requests early
36         http-request deny if { req.body_size gt 104857600 }
37
38     backend api
39         # HTTP options
40         option httpchk GET /health
41         http-check expect status 200
42
43         # Server connection behavior
44         http-reuse aggressive

```



The `http-request deny if { req.body_size gt 104857600 }` line is worth noting: it rejects oversized requests **before** buffering them. Without this, HAProxy would accept a 10GB upload attempt before rejecting it – wasting bandwidth and potentially filling disk.

Scenario	Buffer Setting	Recommendation
Large cookies/headers	<code>client_header_buffer_size</code>	4k-16k
File uploads	<code>client_body_buffer_size</code>	128k-1m
Large API responses	<code>proxy_buffers</code>	16-32 × 32k-64k
Streaming/SSE	<code>proxy_buffering</code>	off
Memory constrained	<code>proxy_max_temp_file_size</code>	Limit spillover

Buffer sizing recommendations by scenario. These are starting points – profile your actual traffic to refine.

INFO

Quick decision: If you're seeing 400 Bad Request errors, increase `client_header_buffer_size`. If backends complain about slow clients, turn `proxy_buffering` on. If you're proxying Server-Sent Events or WebSockets, turn `proxy_buffering` off.

Connection Management

Connection management is where proxies earn their keep. Opening a new TCP (Transmission Control Protocol) connection involves a three-way handshake (50-100ms to a nearby server, 200ms+ cross-region). Add TLS (Transport Layer Security) and you're looking at another round trip or two. At 1000 RPS (Requests Per Second), if every request opens a new backend connection, you're spending more time on handshakes than actual work.



The solution is connection pooling: the proxy maintains a pool of open connections to each backend and reuses them across requests. Nginx calls this “upstream keepalive”; HAProxy calls it “connection reuse.” Both dramatically reduce backend latency.

Keep-Alive Tuning

Nginx’s upstream keepalive has a critical gotcha: it requires specific HTTP headers to work. By default, Nginx sends `Connection: close` to backends, defeating keepalive entirely. You **must** set `proxy_http_version 1.1` and `proxy_set_header Connection ""` in every location block that uses the upstream.

nginx-keepalive.conf

```
1  # === Client-side keep-alive ===
2
3  # Enable keep-alive for clients
4  # Default: on
5  keepalive_timeout 65s;
6
7  # Maximum requests per keep-alive connection
8  # After this many requests, connection closes
9  # Default: 100
10 keepalive_requests 1000;
11
12 # Send keep-alive header
13 # Default: off
14 keepalive_disable none;
15
16 # === Backend keep-alive (upstream) ===
17
18 upstream backend {
19     server backend1:8080;
20     server backend2:8080;
21
22     # Number of idle keep-alive connections to preserve
23     # Per worker process!
24     # Default: none (new connection per request)
25     keepalive 32;
26
27     # Maximum requests per backend keep-alive connection
28     # Default: 1000
29     keepalive_requests 10000;
30
```



```
31     # Idle timeout for backend connections
32     # Default: 60s
33     keepalive_timeout 60s;
34 }
35
36 server {
37     location / {
38         proxy_pass http://backend;
39
40         # Required for upstream keep-alive
41         proxy_http_version 1.1;
42         proxy_set_header Connection "";
43     }
44 }
45
46 # === Connection draining ===
47
48 # Graceful worker shutdown timeout
49 # Wait for connections to finish
50 worker_shutdown_timeout 30s;
```

The `keepalive 32` directive is **per worker process**, not total. With 4 workers, you'll have up to 128 idle backend connections (32×4). Size this based on your request rate and backend count – too few connections means constant churn, too many wastes backend resources holding idle connections.

HAProxy Connection Pooling

HAProxy's `http-reuse` directive controls connection pooling behavior. The `aggressive` setting reuses connections for all HTTP methods, which is safe for stateless backends. The `safe` setting only reuses for idempotent methods (GET, HEAD, OPTIONS)—use this if your backends have connection-affinity issues.

The `maxconn` limits deserve careful thought. The global `maxconn` caps total proxy connections. Frontend `maxconn` caps client connections. Backend server `maxconn` caps connections **per backend server**. When a server hits its limit, HAProxy queues requests (controlled by `timeout queue`) rather than overloading the backend.

haproxy-connections.cfg

```
1  global
```



```

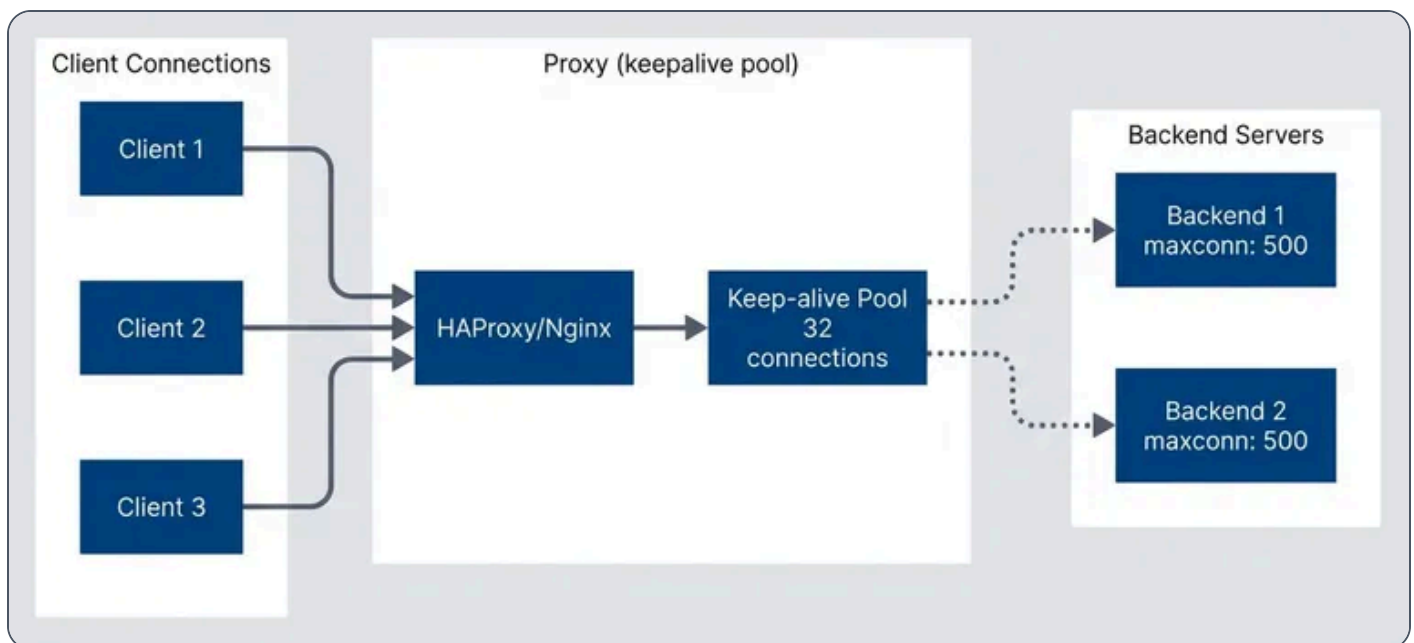
2      # Maximum concurrent connections (global limit)
3      maxconn 50000
4
5      # Number of threads
6      nbthread 4
7
8      defaults
9      mode http
10
11     # Connection reuse policy
12     # safe: reuse only for safe methods
13     # aggressive: reuse for all methods
14     # always: always reuse (dangerous)
15     # never: no reuse
16     option http-reuse aggressive
17
18     # Server connection mode
19     # http-server-close: close backend conn after response
20     # http-keep-alive: keep backend connections open
21     option http-server-close
22
23     frontend main
24     bind *:80
25
26     # Frontend connection limit
27     maxconn 40000
28
29     # Connection rate limiting
30     rate-limit sessions 1000
31
32     default_backend api
33
34     backend api
35     # Load balancing algorithm
36     balance roundrobin
37
38     # Server definitions with connection limits
39     server srv1 backend1:8080 maxconn 500 check
40     server srv2 backend2:8080 maxconn 500 check
41
42     # Queue configuration when servers are full
43     # timeout queue defined in defaults
44     fullconn 800
45

```



```
46     # Connection retry
47     retries 3
48     option redispatch
49
50     # Health checking
51     option httpchk GET /health
52     http-check expect status 200
53
54     # Slow start - ramp up connections to new server
55     server srv3 backend3:8080 maxconn 500 slowstart 30s check
```

The `slowstart` option is valuable when bringing new servers online. Without it, a fresh server immediately receives its full share of traffic – which can overwhelm cold caches and unwarmed JVMs. With `slowstart 30s`, HAProxy gradually increases traffic to the new server over 30 seconds, giving it time to warm up.



Connection pooling to backends.



✓ SUCCESS

Backend keep-alive is often more important than client keep-alive. Each new backend connection requires a TCP (Transmission Control Protocol) handshake (and potentially TLS (Transport Layer Security)). A keep-alive pool dramatically reduces backend latency and load.

Load Balancing Configuration

Load balancing distributes traffic across backend servers. The choice of algorithm matters less than people think – round-robin works fine for most stateless services – but getting it wrong causes real problems. Session affinity (“sticky sessions”) on a stateless service wastes capacity; no affinity on a stateful service causes session corruption.

Algorithm Selection

Round-robin is the default and usually correct for stateless microservices. It distributes traffic evenly regardless of what backends are doing. Least-connections works better when request durations vary significantly – a reporting endpoint that takes 30 seconds shouldn’t receive new requests at the same rate as a health check endpoint.

IP hash provides basic session affinity without cookies, but it’s fragile: clients behind NAT share an IP, mobile clients change IPs frequently, and CDNs mask the original IP. Cookie-based affinity (HAProxy’s `cookie` directive or Nginx Plus’s sticky cookies) is more reliable when you actually need sessions.

nginx-load-balancing.conf

```
1  # Round-robin (default)
2  # Simple rotation through servers
3  upstream backend_rr {
4      server backend1:8080;
5      server backend2:8080;
6      server backend3:8080;
7  }
8
9  # Least connections
```



```
10  # Send to server with fewest active connections
11  upstream backend_lc {
12      least_conn;
13      server backend1:8080;
14      server backend2:8080;
15      server backend3:8080;
16  }
17
18  # IP hash (sticky sessions)
19  # Same client IP goes to same server
20  upstream backend_ip {
21      ip_hash;
22      server backend1:8080;
23      server backend2:8080;
24      server backend3:8080;
25  }
26
27  # Weighted distribution
28  # Higher weight = more traffic
29  upstream backend_weighted {
30      server backend1:8080 weight=5; # 50% of traffic
31      server backend2:8080 weight=3; # 30% of traffic
32      server backend3:8080 weight=2; # 20% of traffic
33  }
34
35  # Health checks and failure handling
36  upstream backend_resilient {
37      server backend1:8080 max_fails=3 fail_timeout=30s;
38      server backend2:8080 max_fails=3 fail_timeout=30s;
39      server backend3:8080 backup; # Only used when others fail
40
41      keepalive 32;
42  }
43
44  # Hash on custom key (e.g., user ID)
45  upstream backend_hash {
46      hash $request_uri consistent;
47      server backend1:8080;
48      server backend2:8080;
49      server backend3:8080;
50  }
```



The `consistent` modifier on hash-based balancing is important for caching use cases. Without it, adding or removing a server rehashes **all** requests to new backends, invalidating caches. With consistent hashing, only requests that were going to the added/removed server get redistributed.

HAProxy Advanced Load Balancing

HAProxy offers more sophisticated options, including agent checks where backends report their own health and capacity. The `random(2)` algorithm (“power of two choices”) is interesting: it picks two servers randomly and sends the request to whichever has fewer connections. This provides near-optimal distribution without the overhead of tracking all server states.

haproxy-load-balancing.cfg

```
1  backend api
2      # === Balancing Algorithms ===
3
4      # Round-robin (default)
5      balance roundrobin
6
7      # Least connections
8      # balance leastconn
9
10     # Source IP hash (sticky)
11     # balance source
12
13     # URI hash (cache efficiency)
14     # balance uri
15
16     # Header-based hash
17     # balance hdr(X-User-ID)
18
19     # Random with power of two choices
20     # balance random(2)
21
22     # === Server weights and options ===
23     server srv1 backend1:8080 weight 100 check
24     server srv2 backend2:8080 weight 100 check
25     server srv3 backend3:8080 weight 50 check # Half traffic
26
27     # === Advanced options ===
28
29     # Dynamic weight adjustment based on health
```




```

30     option httpchk GET /health
31     http-check expect status 200
32     default-server inter 3s fall 3 rise 2
33
34     # Slow start - gradually increase traffic to new/recovered server
35     default-server slowstart 60s
36
37     # Agent check - backend reports own health/weight
38     # server srv1 backend1:8080 check agent-check agent-port 8081
39
40     backend sticky_sessions
41         # Cookie-based sticky sessions
42         balance roundrobin
43         cookie SERVERID insert indirect nocache
44         server srv1 backend1:8080 cookie s1 check
45         server srv2 backend2:8080 cookie s2 check
46
47     backend canary
48         # Weighted canary deployment
49         server stable backend-stable:8080 weight 95 check
50         server canary backend-canary:8080 weight 5 check

```

The canary deployment example shows a common pattern: routing 5% of traffic to a new version while 95% goes to the stable version. If the canary starts failing health checks or returning errors, you can quickly shift traffic back to stable by adjusting weights or marking the canary server as `disabled`.

#	Algorithm	Use Case	Session Affinity	Even Distribution
1	Round-robin	Stateless services	No	Yes
2	Least-conn	Varying request duration	No	Adaptive
3	IP hash	Simple session affinity	Yes (IP-based)	Depends on IPs
4	Cookie	Reliable session affinity	Yes (cookie)	Yes
5	URI hash	Caching proxies	Yes (URL-based)	Depends on URLs



Load balancing algorithm selection guide. Start with round-robin and change only if you have a specific reason.

INFO

For stateless services, `least_conn` often outperforms `round_robin` when request durations vary significantly. It naturally avoids sending requests to servers still processing slow requests.

SSL/TLS Optimization

TLS (Transport Layer Security) termination at the proxy is almost always correct. Backends can run plain HTTP (simpler, faster, easier to debug), while clients get proper encryption. The proxy handles certificate management, cipher negotiation, and session caching – concerns you don't want duplicated across dozens of backend services.

The performance cost of TLS (Transport Layer Security) is often overstated. Modern CPUs have AES-NI¹ instructions that make symmetric encryption nearly free. The real cost is in handshakes: each new connection requires asymmetric cryptography and certificate verification. Session caching and TLS (Transport Layer Security) 1.3's faster handshake mitigate this.

Certificate and Protocol Configuration

TLS (Transport Layer Security) 1.2 is the minimum acceptable version in 2025; TLS (Transport Layer Security) 1.3 is preferred and required by some compliance frameworks. Don't enable TLS (Transport Layer Security) 1.0 or 1.1—they have known vulnerabilities and modern browsers don't use them anyway.

OCSP (Online Certificate Status Protocol) stapling is worth enabling: instead of clients checking certificate revocation with the CA (adding latency), your proxy includes a signed timestamp proving the certificate wasn't revoked. This eliminates a round trip to the CA on each new connection.

Start with certificates and protocol versions:



nginx-ssl-certificates.conf

```

1  server {
2      listen 443 ssl http2;
3      server_name example.com;
4
5      # Certificates
6      ssl_certificate /etc/nginx/ssl/fullchain.pem;
7      ssl_certificate_key /etc/nginx/ssl/privkey.pem;
8
9      # TLS 1.2 minimum (1.3 preferred)
10     ssl_protocols TLSv1.2 TLSv1.3;
11
12     # Prefer server cipher order with modern ciphers
13     ssl_prefer_server_ciphers on;
14     ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-
AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
15 }
```

Session caching reduces TLS (Transport Layer Security) handshake overhead by allowing clients to resume previous sessions. The `shared:SSL:50m` cache is shared across all worker processes:

nginx-ssl-sessions.conf

```

1  server {
2      # Shared cache across workers (50MB)
3      ssl_session_cache shared:SSL:50m;
4      ssl_session_timeout 1d;
5
6      # Session tickets (alternative to cache)
7      ssl_session_tickets on;
8      ssl_session_ticket_key /etc/nginx/ssl/ticket.key;
9  }
10
11 # Generate session ticket key:
12 # openssl rand 80 > /etc/nginx/ssl/ticket.key
```

OCSP (Online Certificate Status Protocol) stapling and DH parameters complete the configuration:



nginx-ssl-ocsp.conf

```
1  server {
2      # OCSP Stapling
3      ssl_stapling on;
4      ssl_stapling_verify on;
5      ssl_trusted_certificate /etc/nginx/ssl/chain.pem;
6      resolver 8.8.8.8 8.8.4.4 valid=300s;
7      resolver_timeout 5s;
8
9      # DH Parameters (generate with: openssl dhparam -out dhparam.pem 2048)
10     ssl_dhparam /etc/nginx/ssl/dhparam.pem;
11
12     # Early data (0-RTT) - careful: replay attacks possible
13     ssl_early_data on;
14     proxy_set_header Early-Data $ssl_early_data;
15 }
```

The `ssl_early_data` directive enables TLS (Transport Layer Security) 1.3's 0-RTT feature, allowing clients to send data on the first packet of a resumed connection. This is genuinely faster but introduces replay attack risk – a captured request could be replayed by an attacker. Only enable this for idempotent endpoints, and have backends check the `Early-Data` header.

HAProxy SSL Configuration

HAProxy can handle multiple certificates from a directory (`cert /etc/haproxy/certs/`) and automatically selects the right one based on SNI (Server Name Indication). This makes certificate management simpler than Nginx's approach of listing each certificate explicitly.

The `ssl verify required ca-file` option for backend connections is important for zero-trust architectures² where even internal traffic should be encrypted and authenticated.

haproxy-ssl.cfg

```
1  global
2      # SSL/TLS defaults
3      ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384
```



```

4      ssl-default-bind-ciphersuites
      TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
5      ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets
6
7      ssl-default-server-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256
8      ssl-default-server-options ssl-min-ver TLSv1.2
9
10     # SSL session cache
11     tune.ssl.cachesize 100000
12     tune.ssl.lifetime 600
13
14     # SSL memory limits
15     tune.ssl.maxrecord 16384
16
17 frontend https
18     bind *:443 ssl crt /etc/haproxy/certs/ alpn h2,http/1.1
19
20     # SNI-based routing
21     use_backend api if { ssl_fc_sni -i api.example.com }
22     use_backend web if { ssl_fc_sni -i www.example.com }
23
24     # HSTS header
25     http-response set-header Strict-Transport-Security "max-age=31536000;
includeSubDomains"
26
27     default_backend web
28
29 backend api
30     # Backend over TLS
31     server srv1 backend1:8443 ssl verify required ca-file /etc/haproxy/ca.pem check
32
33     # Backend with client certificate
34     # server srv1 backend1:8443 ssl crt /etc/haproxy/client.pem verify required ca-file
/etc/haproxy/ca.pem
35
36 frontend tcp_passthrough
37     bind *:8443
38     mode tcp
39
40     # TCP-level SNI inspection
41     tcp-request inspect-delay 5s
42     tcp-request content accept if { req_ssl_hello_type 1 }
43
44     use_backend passthrough if { req_ssl_sni -i passthrough.example.com }

```



The TCP (Transmission Control Protocol) passthrough frontend (`mode tcp`) is useful when you need to route encrypted traffic without terminating TLS (Transport Layer Security). HAProxy inspects just the SNI (Server Name Indication) hostname in the TLS (Transport Layer Security) handshake to make routing decisions, then passes the encrypted connection directly to the backend. This preserves end-to-end encryption when required.

Optimization	Impact	Trade-off
Session cache	Reduces handshakes	Memory usage
OCSP stapling	Faster validation	Requires resolver
TLS 1.3	Faster handshake, better security	Client support
HTTP/2	Multiplexing, fewer connections	Complexity
Early data (0-RTT)	Faster first request	Replay risk

SSL optimization options. Session caching and TLS 1.3 provide the biggest wins with minimal downside.

WARNING

TLS (Transport Layer Security) 1.3's 0-RTT (early data) is vulnerable to replay attacks. Only enable it for idempotent requests (GET), and ensure backends check the `Early-Data` header to handle replays appropriately.

Monitoring and Observability

You can't tune what you can't measure. Before making any production changes, establish baseline metrics: request rate, error rate, latency percentiles (P50, P95, P99 (99th Percentile)), and connection counts. Without these, you're guessing.



Both Nginx and HAProxy expose metrics, but their approaches differ. Nginx's `stub_status` module provides basic connection counts. For detailed metrics, you need the commercial Nginx Plus or the open-source VTS module. HAProxy's built-in stats page is more comprehensive out of the box, showing per-backend health, queue depths, and response time histograms.

Nginx Metrics and Logging

The most valuable Nginx metrics for production debugging are the upstream timing variables.

`$upstream_connect_time` tells you how long it took to establish a connection to the backend – high values here indicate backend TCP (Transmission Control Protocol) backlog issues or network problems.

`$upstream_response_time` is the total time from connection to last byte received. Comparing this to

`$request_time` (total request duration including client delivery) reveals where latency actually occurs.

nginx-monitoring.conf

```
1  # === Status module (basic metrics) ===
2  server {
3      listen 8080;
4
5      location /nginx_status {
6          stub_status on;
7          allow 10.0.0.0/8;
8          deny all;
9      }
10 }
11
12 # Output:
13 # Active connections: 291
14 # server accepts handled requests
15 # 16630948 16630948 31070465
16 # Reading: 6 Writing: 179 Waiting: 106
17
18 # === Enhanced logging ===
19 log_format detailed '$remote_addr - $remote_user [$time_local] '
20                     '"$request" $status $body_bytes_sent '
21                     '"$http_referer" "$http_user_agent" '
22                     'rt=$request_time uct="$upstream_connect_time" '
23                     'uht="$upstream_header_time" urt="$upstream_response_time" '
24                     'cs=$upstream_cache_status';
25
```



```
26 log_format json escape=json '{'
27     '"time":'$time_iso8601','
28     '"remote_addr":'$remote_addr','
29     '"method":'$request_method','
30     '"uri":'$uri','
31     '"status":$status,'
32     '"body_bytes":$body_bytes_sent,'
33     '"request_time":$request_time,'
34     '"upstream_time":$upstream_response_time,'
35     '"upstream_addr":$upstream_addr'
36 '}'
37
38 access_log /var/log/nginx/access.log detailed;
39 # access_log /var/log/nginx/access.json.log json;
40
41 # === Error categorization ===
42 # Log 4xx/5xx to separate file
43 map $status $loggable {
44     ~^[45] 1;
45     default 0;
46 }
47
48 access_log /var/log/nginx/errors.log detailed if=$loggable;
49
50 # === Prometheus exporter ===
51 # Use nginx-prometheus-exporter with stub_status
52 # or nginx-vts-module for detailed metrics
```

The JSON log format is worth adopting early. It's slightly more verbose but makes log analysis with tools like Loki, Elasticsearch, or even `jq` dramatically easier. Structured logs let you query by status code, latency range, or upstream server without regex gymnastics.

INFO

Neither Nginx nor HAProxy natively supports binary log formats. Both write text-based logs (plain or JSON). For high-throughput scenarios where logging becomes a bottleneck, consider writing logs to a RAM disk and rotating frequently, using syslog with UDP transport to an external aggregator, or disabling access logging entirely for health check endpoints.



HAProxy Statistics

HAProxy's stats page is surprisingly powerful. It shows real-time connection counts, queue depths, server health, and response time distributions per backend. The Prometheus exporter endpoint (`/metrics`) makes integrating with modern observability stacks trivial – just point Prometheus at it.

haproxy-monitoring.cfg

```
1  frontend stats
2      bind *:8404
3
4      # Enable stats page
5      stats enable
6      stats uri /stats
7      stats refresh 10s
8      stats auth admin:secure_password
9
10     # Prometheus metrics endpoint
11     http-request use-service prometheus-exporter if { path /metrics }
12
13     # === Built-in metrics ===
14     # Frontend: sessions, bytes, requests, errors
15     # Backend: sessions, bytes, health, queue, response time
16     # Server: sessions, bytes, health, weight, queue
17
18     # === Log format for analysis ===
19     defaults
20         mode http
21         option httplog
22
23         # Detailed log format
24         log-format "%ci:%cp [%tr] %ft %b/%s %TR/%Tw/%Tc/%Tr/%Ta %ST %B %CC %CS %tsc
25         %ac/%fc/%bc/%sc/%rc %sq/%bq %hr %hs %{+Q}r"
26
27         # JSON log format (alternative)
28         # log-format
29         '{ "client": "%ci", "frontend": "%f", "backend": "%b", "server": "%s", "status": %ST, "bytes": %B, "t
30         ime_total": %Ta, "time_response": %Tr } '
31
32     # === Health check logging ===
33     backend api
34         option log-health-checks
```



```
32      server srv1 backend1:8080 check
```

The `%TR/%Tw/%Tc/%Tr/%Ta` timing fields in HAProxy's log format break down request processing: TR is request receive time, Tw is queue wait time, Tc is backend connect time, Tr is backend response time, Ta is total active time. When debugging slow requests, these distinctions matter – a high Tw means your backends are overloaded, while a high Tc means backend TCP (Transmission Control Protocol) accept queues are full.

Alerting on Proxy Metrics

Effective alerts focus on symptoms, not causes. Alert on “5xx error rate above 5%” rather than “nginx worker CPU above 80%.” The error rate directly indicates user impact; the CPU might be high for perfectly acceptable reasons.

prometheus-alerts.yaml

```
1  groups:
2    - name: proxy_alerts
3      rules:
4        - alert: HighErrorRate
5          expr: |
6            sum(rate(nginx_http_requests_total{status=~"5.."}[5m])) /
7            sum(rate(nginx_http_requests_total[5m])) > 0.05
8          for: 5m
9          labels:
10             severity: critical
11          annotations:
12             summary: "High 5xx error rate ({{ $value | humanizePercentage }})"
13
14        - alert: HighLatency
15          expr: |
16            histogram_quantile(0.99,
17              rate(nginx_http_request_duration_seconds_bucket[5m])
18            ) > 2
19          for: 5m
20          labels:
21             severity: warning
22          annotations:
23             summary: "P99 latency above 2 seconds"
24
```



```
25     - alert: BackendDown
26       expr: haproxy_server_up == 0
27       for: 1m
28       labels:
29         severity: critical
30       annotations:
31         summary: "Backend server {{ $labels.server }} is down"
32
33     - alert: HighConnectionCount
34       expr: nginx_connections_active > 10000
35       for: 5m
36       labels:
37         severity: warning
38       annotations:
39         summary: "High active connection count"
40
41     - alert: HighQueueDepth
42       expr: haproxy_backend_current_queue > 100
43       for: 2m
44       labels:
45         severity: warning
46       annotations:
47         summary: "Backend queue depth high"
```

Prometheus alerting rules for proxy health monitoring.

The queue depth alert (`haproxy_backend_current_queue > 100`) is particularly valuable. A growing queue means backends can't keep up with incoming traffic – you're either under-provisioned or experiencing a backend problem. This often precedes user-visible errors by minutes, giving you time to react.

✓ SUCCESS

Log upstream timing (`$upstream_response_time`) separately from total request time (`$request_time`). The difference reveals how much time is spent in the proxy itself – useful for debugging proxy overhead vs. backend slowness.



Production Hardening Checklist

Beyond performance tuning, production proxies need security hardening. A misconfigured proxy can leak internal server information, allow request smuggling attacks, or become an amplifier for DDoS attacks.

Security Configuration

Start with the basics: hide version information (`server_tokens off`), add security headers, and implement rate limiting. Rate limiting at the proxy layer is your first line of defense against abuse – it protects backends from being overwhelmed by a single misbehaving client.

nginx-security.conf

```
1  # === Hide version ===
2  server_tokens off;
3
4  # === Security headers ===
5  add_header X-Frame-Options "SAMEORIGIN" always;
6  add_header X-Content-Type-Options "nosniff" always;
7  add_header Referrer-Policy "strict-origin-when-cross-origin" always;
8
9  # HSTS (enable after testing)
10 add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
11
12 # === Rate limiting ===
13 limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
14 limit_conn_zone $binary_remote_addr zone=conn:10m;
15
16 server {
17     location /api/ {
18         limit_req zone=api burst=20 nodelay;
19         limit_conn conn 10;
20
21         proxy_pass http://backend;
22     }
23 }
24
25 # === Request filtering ===
26 # Block common attacks
27 location ~* "(base64_encode|eval|\\(|javascript:)" {
28     return 403;
```



```
29  }
30
31  # Block sensitive paths
32  location ~ /\. {
33      deny all;
34  }
35
36  # === Client restrictions ===
37  # Maximum header size
38  large_client_header_buffers 4 8k;
39
40  # Maximum body size
41  client_max_body_size 10m;
42
43  # Maximum URI length
44  # Handled by large_client_header_buffers
```

The security headers serve distinct purposes. `X-Frame-Options: SAMEORIGIN` prevents clickjacking by blocking your pages from being embedded in iframes on other domains. `X-Content-Type-Options: nosniff` stops browsers from MIME-sniffing responses away from their declared content type – a common vector for XSS attacks. `Strict-Transport-Security` (HSTS) tells browsers to only connect via HTTPS for the specified duration; once set, even typing `http://` will redirect to HTTPS before the request leaves the browser.

The `limit_req` directive with `burst` and `nodelay` is the most useful rate limiting pattern. The base rate (10r/s in this example) controls sustained throughput, while the burst (20) allows brief spikes without rejecting requests. The `nodelay` option serves burst requests immediately rather than spacing them out.

Complete Production Config Example

Here's a complete Nginx configuration incorporating everything discussed. This isn't copy-paste ready – your values will differ based on traffic patterns, backend capabilities, and compliance requirements – but it demonstrates how the pieces fit together.

Start with the main context and events block. The `worker_rlimit_nofile` setting increases the file descriptor limit per worker, essential when handling many concurrent connections:



nginx-production-main.conf

```
1 worker_processes auto;
2 worker_rlimit_nofile 65535;
3 pid /run/nginx.pid;
4
5 events {
6     worker_connections 4096;
7     use epoll;
8     multi_accept on;
9 }
```

The http block contains global settings that apply to all virtual hosts. Timeouts and buffers here serve as defaults that individual locations can override:

nginx-production-http.conf

```
1 http {
2     # Basic settings
3     sendfile on;
4     tcp_nopush on;
5     tcp_nodelay on;
6     types_hash_max_size 2048;
7     server_tokens off;
8
9     include /etc/nginx/mime.types;
10    default_type application/octet-stream;
11
12    # Logging with upstream timing
13    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
14                    '$status $body_bytes_sent "$http_referer" '
15                    '"$http_user_agent" rt=$request_time '
16                    'uct=$upstream_connect_time uht=$upstream_header_time '
17                    'urt=$upstream_response_time';
18
19    access_log /var/log/nginx/access.log main buffer=16k flush=2m;
20    error_log /var/log/nginx/error.log warn;
21
22    # Timeouts
23    client_header_timeout 10s;
24    client_body_timeout 30s;
```



```

25     send_timeout 30s;
26     keepalive_timeout 65s;
27     keepalive_requests 1000;
28
29     # Buffers
30     client_header_buffer_size 4k;
31     large_client_header_buffers 8 16k;
32     client_body_buffer_size 128k;
33     client_max_body_size 100m;
34
35     # Proxy defaults
36     proxy_connect_timeout 5s;
37     proxy_send_timeout 30s;
38     proxy_read_timeout 60s;
39     proxy_buffer_size 8k;
40     proxy_buffers 16 32k;
41     proxy_busy_buffers_size 64k;
42
43     # Compression
44     gzip on;
45     gzip_vary on;
46     gzip_proxied any;
47     gzip_comp_level 5;
48     gzip_types text/plain text/css application/json application/javascript text/xml
application/xml;
49
50     # Rate limiting zones
51     limit_req_zone $binary_remote_addr zone=api:10m rate=100r/s;
52     limit_conn_zone $binary_remote_addr zone=conn:10m;
53 }
```

The upstream and server blocks define the backend pool and virtual host. Note the `keepalive 32` on the upstream and the required `proxy_http_version 1.1` with empty `Connection` header to enable backend connection reuse:

nginx-production-server.conf

```

1 http {
2     upstream backend {
3         least_conn;
4         server backend1:8080 max_fails=3 fail_timeout=30s;
```



```

5      server backend2:8080 max_fails=3 fail_timeout=30s;
6      keepalive 32;
7      keepalive_requests 10000;
8      keepalive_timeout 60s;
9  }
10
11  server {
12      listen 443 ssl http2;
13      server_name example.com;
14
15      # SSL
16      ssl_certificate /etc/nginx/ssl/fullchain.pem;
17      ssl_certificate_key /etc/nginx/ssl/privkey.pem;
18      ssl_protocols TLSv1.2 TLSv1.3;
19      ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;
20      ssl_prefer_server_ciphers on;
21      ssl_session_cache shared:SSL:50m;
22      ssl_session_timeout 1d;
23      ssl_stapling on;
24      ssl_stapling_verify on;
25
26      # Security headers
27      add_header X-Frame-Options "SAMEORIGIN" always;
28      add_header X-Content-Type-Options "nosniff" always;
29      add_header Strict-Transport-Security "max-age=31536000" always;
30
31      location / {
32          limit_req zone=api burst=50 nodelay;
33          limit_conn conn 20;
34
35          proxy_pass http://backend;
36          proxy_http_version 1.1;
37          proxy_set_header Connection "";
38          proxy_set_header Host $host;
39          proxy_set_header X-Real-IP $remote_addr;
40          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
41          proxy_set_header X-Forwarded-Proto $scheme;
42      }
43
44      location /health {
45          access_log off;
46          proxy_pass http://backend;
47          proxy_connect_timeout 2s;

```




```

48         proxy_read_timeout 5s;
49     }
50 }
51 }
```

Note the `buffer=16k flush=2m` on the access log – this buffers log writes to reduce disk I/O, flushing every 2 minutes or when the buffer fills. At high request rates, unbuffered logging becomes a bottleneck.

#	Category	Default	Production	Why
1	<code>worker_connections</code>	1024	4096+	Handle more concurrent connections
2	<code>keepalive_timeout</code>	75s	65s	Balance reuse vs. resource usage
3	<code>proxy_connect_timeout</code>	60s	5s	Fail fast on unreachable backends
4	<code>proxy_buffers</code>	8 4k	16 32k	Handle larger responses
5	<code>ssl_session_cache</code>	none	shared:50m	Reduce TLS handshakes

Default vs. production values. These are starting points – profile your actual traffic to determine optimal values.

⚠ DANGER

Before deploying configuration changes to production, test under realistic load. Use tools like `wrk`, `hey`, or `k6` to simulate traffic patterns. Configuration that works at 100 RPS (Requests Per Second) may fail at 10,000 RPS (Requests Per Second).

Applying Configuration Changes

Both Nginx and HAProxy support graceful reloads that apply new configurations without dropping active connections. For Nginx, `nginx -s reload` spawns new workers with the new config while existing workers finish their current requests. HAProxy's `systemctl reload haproxy` (or `-sf` flag for seamless reload)



does the same. In Kubernetes, the ingress controller handles this automatically when ConfigMaps change.

The key is testing configuration syntax **before** reloading. A syntax error during reload can leave you with no running workers. Always run `nginx -t` or `haproxy -c -f /etc/haproxy/haproxy.cfg` first.

Conclusion

Proxy defaults are starting points, not production configurations. Nginx's 60-second timeouts and HAProxy's unlimited defaults exist to avoid breaking things during development – they're the wrong choices for production.

The tuning process follows a pattern: establish baseline metrics, identify bottlenecks, adjust configuration, measure again. Timeouts should match your traffic patterns – slow report endpoints need longer read timeouts, health checks need aggressive timeouts that fail fast. Buffers should accommodate your payloads without wasting memory. Connection pools should be sized for your request rate and backend count.

The goal isn't a perfectly optimized configuration – it's a **resilient** one. Your proxy should handle normal traffic with good performance, absorb traffic spikes without dropping connections, and shed load gracefully when backends struggle. Test under failure conditions: simulate slow backends, connection storms, and oversized payloads. The problems you find in staging won't page you at 3 AM.

-
- 1 Intel AES-NI (Advanced Encryption Standard New Instructions) is a hardware-based extension for x86 processors (Intel and AMD) designed to significantly speed up data encryption and decryption. It uses specific CPU instructions to handle complex cryptographic calculations, providing 3x to 10x faster performance while reducing CPU load compared to software-only implementations. ↪
 - 2 Zero-trust architecture is a security model that assumes no implicit trust for any user, device, or network – even within the corporate perimeter. Every request must be authenticated, authorized, and encrypted regardless of where it originates. In practice, this means TLS (Transport Layer Security) everywhere (not just at the edge), mutual TLS (Transport Layer Security) between services, and continuous verification rather than one-time authentication at the network boundary. ↪



Copyright © 2024 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/nginx-haproxy-reverse-proxy-production-tuning>

